

Document number	P0439R0
Date	2016-10-06
Project	Programming Language C++, Library Evolution Working Group, SG1
Reply-to	Jonathan Wakely <cxx@kayari.org>

Make `std::memory_order` a scoped enumeration

Abstract

I propose changing `std::memory_order` to be a scoped enumeration type, in order to improve type safety and support more idiomatic C++ syntax.

Background

The enumeration type `std::memory_order` is defined in 29.3 [atomics.order] as:

```
namespace std {
    typedef enum memory_order {
        memory_order_relaxed, memory_order_consume, memory_order_acquire,
        memory_order_release, memory_order_acq_rel, memory_order_seq_cst
    } memory_order;
}
```

This style was used so the definition would be compatible with C, but that noble aim failed with the introduction of `_Atomic` as a type qualifier in C11. In any case, the fact it's defined in namespace `std` means it is a distinct type from an equivalent type defined in the global namespace.

The style of specification (as a typedef for an enumeration type of the same name) is entirely unnecessary in C++, and the naming of the enumerators is an unfortunate wart. Let's address it.

What `memory_order` would look like in idiomatic C++

There is no reason to continue specifying it as a typedef-declaration, and we should editorially change the specification to:

```
namespace std {
    typedef enum memory_order {
        memory_order_relaxed, memory_order_consume, memory_order_acquire,
        memory_order_release, memory_order_acq_rel, memory_order_seq_cst
    } memory_order;
}
```

But let's not stop there. If I was designing this type today I would not have named the enumerators with the type name as a prefix, I would have used a scoped enumeration type:

```
enum class memory_order {
    relaxed, consume, acquire, release, acq_rel, seq_cst
};
```

To name an enumerator you would say `memory_order::acquire` instead of `memory_order_acquire`, or following `using MO = std::memory_order`; you could say `MO::acquire` instead.

More importantly, a scoped enumeration is more strongly typed, disallowing implicit conversion to integers, and preventing nonsensical expressions such as `memory_order_acquire|memory_order_release` (which could be mistakenly assumed to be equivalent to `memory_order_acq_rel`) or `~memory_order_relaxed`. A scoped enumeration is a better match for the desired semantics of these constants.

Compatibility

In order to maintain source compatibility we can declare inline variables of the enumeration type with the existing names:

```
inline constexpr auto memory_order_relaxed = memory_order::relaxed;
inline constexpr auto memory_order_consume = memory_order::consume;
inline constexpr auto memory_order_acquire = memory_order::acquire;
inline constexpr auto memory_order_release = memory_order::release;
inline constexpr auto memory_order_acq_rel = memory_order::acq_rel;
inline constexpr auto memory_order_seq_cst = memory_order::seq_cst;
```

This ensures that any source code using the existing names continues to compile and has the same meaning.

In order to maintain binary compatibility the underlying type of the scoped enumeration type should be left unspecified, so that implementations can set it to match the underlying type that was previously chosen (either implicitly or explicitly) for the unscoped enumeration type.

For the Itanium C++ ABI (and to the best of my knowledge the VC++ ABI) the `memory_order` type is mangled the same whether it's a scoped or unscoped enumeration type. I don't believe the proposed changes to the enumerator names (and introduction of inline variables for the old names) can change the mangled names of any symbols either. The old names were only enumerators, so they are not lvalues and only their values (not names) can appear in mangled names.

Alternative

Since C++11 it has been possible to refer to enumerators of unscoped enumerations using the type name as a nested-name-qualifier. In order to support `memory_order::seq_cst` we could simply add extra enumerators:

```
enum memory_order {
    memory_order_relaxed, memory_order_consume, memory_order_acquire,
    memory_order_release, memory_order_acq_rel, memory_order_seq_cst,
    relaxed = memory_order_relaxed,
    consume = memory_order_consume,
    acquire = memory_order_acquire,
    release = memory_order_release,
    acq_rel = memory_order_acq_rel,
    seq_cst = memory_order_seq_cst
};
```

This doesn't have the type safety advantages of a scoped enumeration, and the new enumerators would "leak" into the surrounding namespace, so would also be visible as `std::seq_cst`. This alternative would not be an improvement.

Related Issues

`std::atomic_flag` is also defined as a typedef declaration, and relying on the preprocessor for `ATOMIC_FLAG_INIT` is most foul.

Proposed Wording

```
namespace std {
    typedef enum class memory_order : unspecified {
        memory_order_relaxed, memory_order_consume, memory_order_acquire,
        memory_order_release, memory_order_acq_rel, memory_order_seq_cst
        relaxed, consume, acquire, release, acq_rel, seq_cst
    } memory_order;
    inline constexpr memory_order memory_order_relaxed = memory_order::relaxed;
    inline constexpr memory_order memory_order_consume = memory_order::consume;
    inline constexpr memory_order memory_order_acquire = memory_order::acquire;
    inline constexpr memory_order memory_order_release = memory_order::release;
    inline constexpr memory_order memory_order_acq_rel = memory_order::acq_rel;
    inline constexpr memory_order memory_order_seq_cst = memory_order::seq_cst;
}
```

The enumeration `memory_order` specifies the detailed regular (non-atomic) memory synchronization order as defined in 1.10 and may provide for operation ordering. Its enumerated values and their meanings are as follows: — `memory_order::relaxed`: no operation orders memory. — `memory_order::release`, `memory_order::acq_rel`, and `memory_order::seq_cst`: a store operation performs a release operation on the affected memory location. — `memory_order::consume`: a load operation performs a consume operation on the affected memory location. [Note: Prefer `memory_order::acquire`, which provides stronger guarantees than `memory_order::consume`. Implementations have found it infeasible to provide performance better than that of `memory_order::acquire`. Specification revisions are under consideration. — end note] — `memory_order::acquire`, `memory_order::acq_rel`, and `memory_order::seq_cst`: a load operation performs an acquire operation on the affected memory location.

Acknowledgments

Thanks to Maurice Bos for helping to check the ABI consequences of the change.